The University of Azad Jammu and Kashmir,

Muzaffarabad

Name	Kamal Ali Akmal
Course Name	Data Structure & Algorithm
Submitted to	Engr. Asma Javaid
Semester	3rd
Session	2024-2028
Roll No	2024-SE-38
Lab No	07
Submission date	08 January 2026

Department of Software Engineering

What is a Queue?

A Queue is a linear data structure that follows the FIFO (First In, First Out) principle. This means the first element added to the queue will be the first one to be removed, similar to a real-world queue (like a line of people waiting at a ticket counter).

Linear Queue (Array-Based)

- Uses a **fixed-size array** to store elements.
- Maintains two pointers:
  - **front**: points to the first element
  - **rear**: points to the last element
- **Drawback**: Once the rear reaches the end of the array, no more elements can be added, even if space is available at the front (due to dequeued elements). This is called **"memory wastage"**.

Lab_07_DSA_2024-SE-38.cpp

```cpp
////   TASK 01 ( Queue Using Array (Linear Queue) )
#include <iostream>
using namespace std;

class Queue {
    int arr[100];
    int front, rear;
public:
    Queue() {
        front = rear = -1;
    }

    bool isEmpty() {
        return (front == -1 || front > rear);
    }

    bool isFull() {
        return (rear == 99);
    }

    void enqueue(int value) {
        if (isFull()) {
            cout << "Queue is full! Cannot enqueue " << value << endl;
            return;
        }
        if (front == -1) front = 0;
        arr[++rear] = value;
    }

    void dequeue() {
        if (isEmpty()) {
            cout << "Queue is empty! Cannot dequeue" << endl;
            return;
        }
        front++;
    }

    int peek() {
```

```
        if (isEmpty()) {
            cout << "Queue is empty!" << endl;
            return -1;
        }
        return arr[front];
    }
};

int main() {
    Queue q;

    // Simple test cases
    q.enqueue(10);
    q.enqueue(20);
    q.enqueue(30);

    cout << "Front element: " << q.peek() << endl;

    q.dequeue();
    cout << "Front element after dequeue: " << q.peek() << endl;

    q.dequeue();
    q.dequeue();

    // Test empty queue
    cout << "Testing empty queue peek: ";
    q.peek();

    return 0;
}
```

```
D:\UNIVRERSITY\3RD SEMEST

Front element: 10
Front element after dequeue: 20
Testing empty queue peek: Queue is empty!

--------------------------------
Process exited after 0.03459 seconds with return value 0
Press any key to continue . . .
```

Circular Queue

- Also uses a **fixed-size array**, but treats it as a **circular buffer**.
- When the rear pointer reaches the end, it wraps around to the beginning if space is available.
- **Benefit**: Eliminates the memory wastage problem of linear queues.
- Uses **modulo arithmetic** for pointer updates:

[*] Lab_07_DSA_2024-SE-38.cpp

```cpp
//    TASK 02 ( Circular Queue )

#include <iostream>
using namespace std;

class CircularQueue {
    int arr[100];
    int front, rear;
    int capacity;
public:
    CircularQueue() {
        front = rear = -1;
        capacity = 100;
    }

    bool isFull() {
        return ((rear + 1) % capacity == front);
    }

    bool isEmpty() {
        return (front == -1);
    }

    void enqueue(int value) {
        if (isFull()) return;
        if (isEmpty()) {
            front = rear = 0;
        } else {
            rear = (rear + 1) % capacity;
        }
        arr[rear] = value;
    }

    void dequeue() {
        if (isEmpty()) return;
        if (front == rear) {
            front = rear = -1;
        } else {
            front = (front + 1) % capacity;
        }
    }

    int peek() {
        if (isEmpty()) return -1;
        return arr[front];
    }
};

int main() {
    CircularQueue cq;

    // Simple operations
    cq.enqueue(5);
    cq.enqueue(10);
    cq.enqueue(15);

    cout << "Front: " << cq.peek() << endl;  // 5

    cq.dequeue();
    cout << "After dequeue, Front: " << cq.peek() << endl;  // 10

    cq.enqueue(20);
    cq.enqueue(25);

    cout << "Front after more enqueues: " << cq.peek() << endl;  // 10

    return 0;
}
```

```
D:\UNIVRERSITY\3RD SEMEST

Front: 5
After dequeue, Front: 10
Front after more enqueues: 10

--------------------------------
Process exited after 0.238 seconds with return value 0
Press any key to continue . . .
```

Queue Using Linked List

- Uses **dynamic memory allocation** (nodes) to store elements.
- Each node contains:
  - **data**: the element value
  - **next**: pointer to the next node
- **Front** pointer points to the first node, **Rear** pointer points to the last node.
- **Advantages**:
  - No fixed size limit
  - Efficient memory usage (allocates only what's needed)
- **Disadvantages**:
  - Extra memory for storing pointers
  - Slightly slower due to dynamic allocation

[*] Lab_07_DSA_2024-SE-38.cpp

```cpp
//   TASK 03 ( Queue Using Linked List )
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
};

class Queue {
    Node* front;
    Node* rear;

public:
    Queue() {
        front = rear = nullptr;
    }

    void enqueue(int value) {
        Node* temp = new Node();
        temp->data = value;
        temp->next = nullptr;

        if (rear == nullptr) {
            front = rear = temp;
        } else {
            rear->next = temp;
            rear = temp;
        }
    }

    void dequeue() {
        if (front == nullptr) return;

        Node* temp = front;
        front = front->next;

        if (front == nullptr) rear = nullptr;
```

[*] Lab_07_DSA_2024-SE-38.cpp

```
176
177         delete temp;
178     }
179
180     int peek() {
181         if (front == nullptr) return -1;
182         return front->data;
183     }
184
185     bool isEmpty() {
186         return front == nullptr;
187     }
188 };
189
190 int main() {
191     Queue q;
192
193     // Test basic operations
194     q.enqueue(10);
195     q.enqueue(20);
196     q.enqueue(30);
197
198     cout << "Front element: " << q.peek() << endl;  // 10
199
200     q.dequeue();
201     cout << "After dequeue, Front element: " << q.peek() << endl;  // 20
202
203     q.dequeue();
204     q.dequeue();
205
206     cout << "After all dequeues, Front element: " << q.peek() << endl;  // -1
207     cout << "Is queue empty? " << (q.isEmpty() ? "Yes" : "No") << endl;  // Yes
208
209     return 0;
210 }
```

D:\UNIVRERSITY\3RD SEMEST

```
Front element: 10
After dequeue, Front element: 20
After all dequeues, Front element: -1
Is queue empty? Yes

--------------------------------
Process exited after 0.244 seconds with return value 0
Press any key to continue . . .
```